

POLYNIAL REGRESSION

1. THE BASIC IDEA

In **Linear Regression**, we assume the relationship between **X** and **Y** is approximately a straight line:

$$y = \beta_0 + \beta_1 x$$

But what if the relationship looks **curved**?

For example:

- Experience $\uparrow \rightarrow$ Salary $\uparrow$, but not at a constant rate
- Temperature $\rightarrow$ Electricity consumption
- Age $\rightarrow$ Healthcare cost
- Speed $\rightarrow$ Fuel consumption

A straight line may not capture the pattern well.

Polynomial Regression lets us model that **curvature**.

2. THE EQUATION

For degree 2:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2$$

For degree 3:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$$

Generally:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_n x^n$$

Here, **n** = **degree of the polynomial**.

3. THE IMPORTANT NOTE

Polynomial Regression is actually built on top of **Linear Regression**.

Suppose:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2$$

We can create new features:

X	X ²
---	----------------

2	4
---	---

3	9
---	---

4	16
---	----

5	25
---	----

Then Linear Regression learns:

$$y = \beta_0 + \beta_1(X) + \beta_2(X^2)$$

So the model is **linear in its coefficients**, even though the relationship with **X** is curved.

4. EXAMPLE

Suppose we have:

Experience	Salary
------------	--------

1	30
---	----

2	40
---	----

3 55

4 75

5 100

A linear model might struggle because salary isn't increasing at a constant rate.

Polynomial Regression can learn something like:

$$\text{Salary} = \beta_0 + \beta_1 \text{Experience} + \beta_2 \text{Experience}^2$$

The **Experience²** feature allows the model to capture the curvature.

5. DEGREE MATTERS

This is **very important** for interviews and practical ML.

Degree 1

$$y = \beta_0 + \beta_1 X$$

Basically **Linear Regression**.

Degree 2

$$y = \beta_0 + \beta_1 X + \beta_2 X^2$$

Quadratic curve.

Degree 3

$$y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3$$

More flexible curve.

Very high degree

The model can become **too flexible** and start fitting noise.

That's **overfitting**.

So:

Low degree → may underfit

Appropriate degree → good fit

Very high degree → may overfit

The crux

Remember this one sentence:

Polynomial Regression = Linear Regression + Polynomial Features.

6. PRACTICAL DONE IN NOTEBOOK IN THE SAME ACCOUNT: suyaladitya915@gmail.com

7. SMJHNE K LIYE RAW MATHEMATICAL SIMULATION ON A SAMPLE DATA

Ab hum **Polynomial Regression ko bilkul scratch se**, ek chhote mathematical example ke saath samjhenge — including:

data → nonlinearity → train/test → polynomial features → prediction → error → gradient → parameter update → `.fit()` actually kya karta hai → final curve

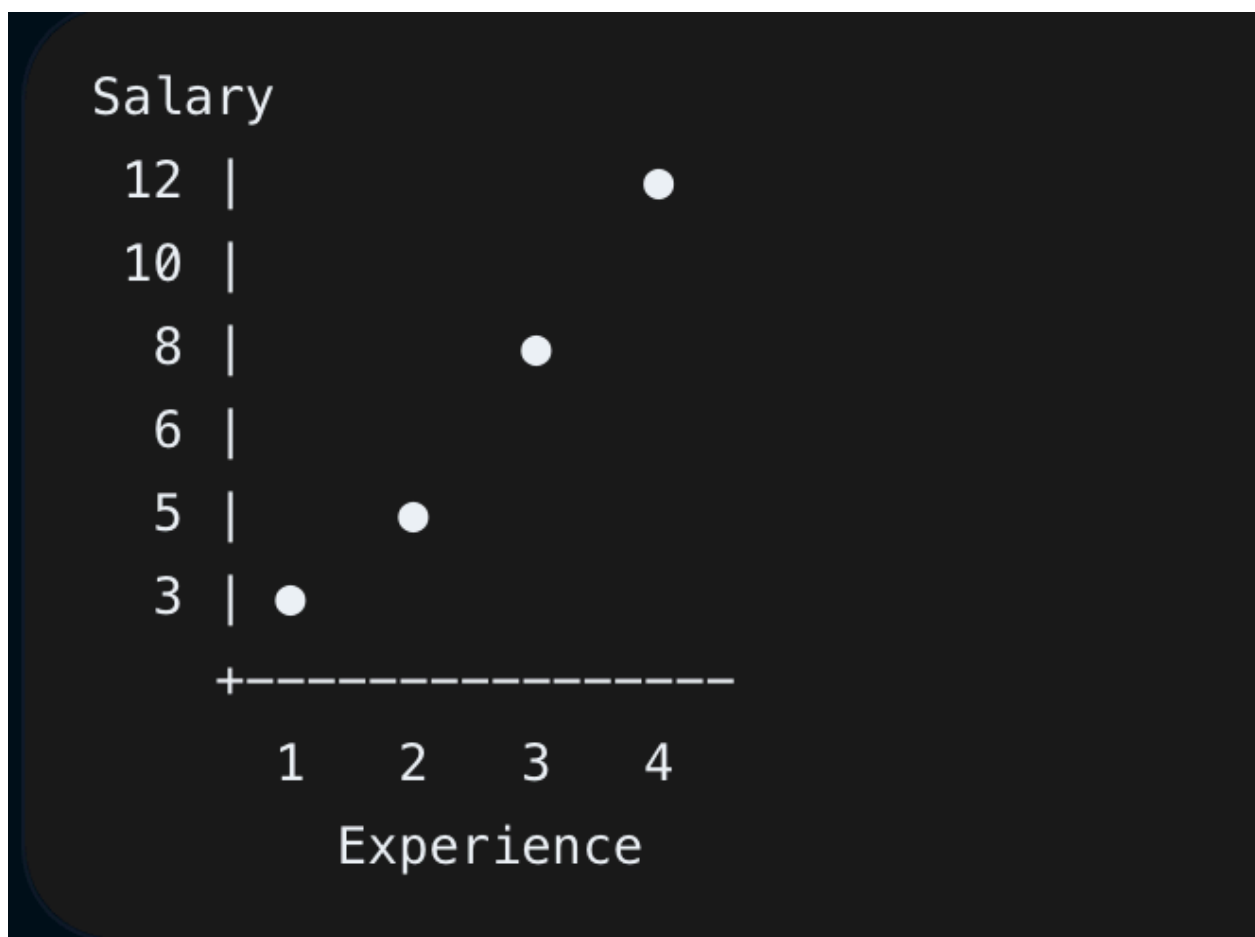
7.1 Sabse pehle: Data aaya

Maan lo hamare paas ek tiny dataset hai:

Experience X **Salary y**

1	3
2	5
3	8
4	12

Agar hum scatter plot karein, pattern kuch aisa hai:



Dekho:

X increase ho raha hai

↓

y bhi increase ho raha hai

↓

but constant rate se nahi

↓

relationship curved/non-linear lag rahi hai

Agar hum Linear Regression lagayenge:

$$y = \theta_0 + \theta_1 x$$

toh model sirf **straight line** bana sakta hai.

7.3 Train/Test split

Real ML mein hum saara data model ko nahi dete.

Suppose:

Training:

$$X = [1, 2, 3]$$

$$y = [3, 5, 8]$$

Testing:

$$X = [4]$$

$$y = [12]$$

Model ko $x=4 \rightarrow y=12$ pehle se nahi pata.

Ye important hai because hume check karna hai:

Model ne pattern actually learn kiya hai ya training examples memorize kiye hain?

4. Polynomial Regression ka main trick



Ab hum original feature:

x

ko transform karenge:

x, x^2

So:

x	x^2
-----	-------

1	1
---	---

2	4
---	---

3	9
---	---

4	16
---	----

Ab model ko ye features milenge:

$[1, x, x^2]$

Aur model learn karega:

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2$$

Yehi Polynomial Regression ka core hai.

5. Important twist 🤖

Tum soch sakte ho:

"Agar equation nonlinear hai, toh model nonlinear ho gaya?"

Input ke respect mein relationship nonlinear hai.

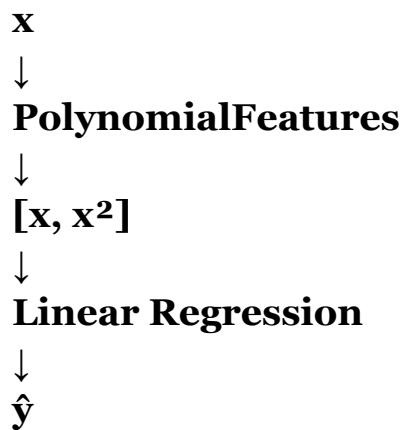
But parameters ke respect mein:

$\theta_0, \theta_1, \theta_2$

equation **linear** hai.

Isliye hum still `LinearRegression()` use kar sakte hain.

Pipeline:



6. Ab actual training start hoti hai

Let's assume initially model ko kuch bhi nahi pata.

Hum initialize karte hain:

$$\theta_0 = 0$$

$$\theta_1 = 0$$

$$\theta_2 = 0$$

So model initially:

$$\hat{y} = 0 + 0x + 0x^2$$

Therefore:

$$\hat{y} = 0$$

for every point.

7. First prediction

Training data:

x	Actual y	Prediction
1	3	0
2	5	0
3	8	0

Errors:

$$\text{error} = \hat{y} - y$$

So:

$$x=1 \rightarrow 0 - 3 = -3$$

$$x=2 \rightarrow 0 - 5 = -5$$

$$x=3 \rightarrow 0 - 8 = -8$$

8. Error ko ek number mein convert karna

Usually Mean Squared Error:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y^i - y_i)^2$$

For our **full 4-point illustration**:

Errors:

-3

-5

-8

-12

Squares:

9

25

64

144

Sum:

$$9 + 25 + 64 + 144 = 242$$

Average:

$$MSE = \frac{242}{4} = 60.5$$

So initially:

$$MSE = 60.5$$

That's terrible 😊 — model knows nothing.

9. Ab question: parameters ko kaise improve karein?

We have:

$MSE(\theta_0, \theta_1, \theta_2)$

We want:

$MSE \downarrow$

So hume pata karna hai:

Kaunsa parameter kis direction mein change karna chahiye?

Yahan aata hai **gradient**.

10. Gradient kya karta hai?

Gradient hume batata hai:

"Idhar jaoge toh error increase hoga."

So hum opposite direction mein move karte hain:

$$\theta_{new} = \theta_{old} - \alpha \frac{\partial MSE}{\partial \theta}$$

where:

$\alpha = \text{learning rate}$

11. Polynomial Regression ka gradient

Our model:

$$\hat{y}_i = \theta_0 + \theta_1 x_i + \theta_2 x_i^2$$

MSE:

$$MSE = \frac{1}{n} \sum (\hat{y}_i - y_i)^2$$

Gradients:

$$\frac{\partial MSE}{\partial \theta_0} = \frac{2}{n} \sum (\hat{y}_i - y_i)$$

$$\frac{\partial MSE}{\partial \theta_1} = \frac{2}{n} \sum (\hat{y}_i - y_i) x_i$$

$$\frac{\partial MSE}{\partial \theta_2} = \frac{2}{n} \sum (\hat{y}_i - y_i) x_i^2$$

12. Let's actually calculate ONE gradient step 🔥

Initially:

$$\theta = [0, 0, 0]$$

Predictions:

$$\hat{y} = [0, 0, 0, 0]$$

Errors:

$$[-3, -5, -8, -12]$$

Now:

Gradient for θ_0

$$2/4(-3-5-8-12)=-14$$

So:

$$g_0=-14$$

Gradient for θ_1

$$2/4[(-3)(1)+(-5)(2)+(-8)(3)+(-12)(4)]$$

Inside:

$$-3-10-24-48=-85$$

Therefore:

$$g_1=-42.5$$

Gradient for θ_2

$$2/4[(-3)(12)+(-5)(22)+(-8)(32)+(-12)(42)]=12[-3-20-72-192]=-143.5$$

So our gradient is:

$$[-14, -42.5, -143.5]$$

13. Parameters update

Let's use:

$$\text{learning rate}=0.001$$

Formula:

$$\theta_{\text{new}}=\theta_{\text{old}}-\alpha \text{gradient}$$

Therefore:

$$\theta_0$$

$$0 - (0.001)(-14) = 0.014$$

$$\theta_1$$

$$0 - (0.001)(-42.5) = 0.0425$$

$$\theta_2$$

$$0 - (0.001)(-143.5) = 0.1435$$

So after **one iteration**:

$$\theta = [0.014, 0.0425, 0.1435]$$

Our model changed from:

$$y = 0$$

to:

$$\hat{y} = 0.014 + 0.0425x + 0.1435x^2$$

14. Now predict again

For $x = 1$:

$$\hat{y} = 0.014 + 0.0425(1) + 0.1435(1) = 0.20$$

For $x = 2$:

$$\hat{y} = 0.014 + 0.085 + 0.574 = 0.673$$

For $x = 3$:

$$\hat{y} = 0.014 + 0.1275 + 1.2915 = 1.433$$

For $x = 4$:

$$\hat{y} = 0.014 + 0.17 + 2.296 = 2.48$$

Still bad

But notice:

Before:

$$\text{MSE} = 60.5$$

After ONE step:

$$\text{MSE} \approx 40.08$$

Error decreased!

That's the entire fundamental idea behind gradient descent.

15. Then repeat

Training doesn't stop after one update.

We repeat:

1. Predict



2. Calculate error



3. Calculate MSE



4. Calculate gradients



5. Update θ



6. Predict again



7. Repeat

Conceptually:

Iteration 0 → MSE 60.50

Iteration 1 → MSE 40.08

Iteration 2 → MSE 26.76

Iteration 5 → MSE 8.72

Iteration 10 → MSE 2.59

Iteration 20 → MSE 1.74

...

Eventually error approaches its minimum.